

Plan: Zoho Books + Bank API Integration (SBI & Canara)

Context

This hospital management system (Adamrit HMS) currently syncs financial data to Tally (on-premise, XML-based). The user wants to:

1. Add Zoho Books cloud accounting as a second accounting integration
2. Add SBI and Canara Bank API integration for live bank balance and statement reconciliation

Both features follow patterns already established for Tally. The bank integration feeds into the existing bank reconciliation UI.

Architecture Overview

Frontend (ZohoBooksPage) -> /api/zoho-books-proxy -> Zoho Books REST API

?

Supabase (config, sync log, contact mapping)

?

Auto-push on bill/payment save (zoho-books-auto-push.ts)

What Gets Synced -- Detailed Field Mappings

1. Patient Bill -> Zoho Books Sales Invoice

Trigger: Bill finalized ([pushBillToZoho])

HMS Field	Zoho Books Field	Notes
[bill.billNumber]	[reference_number]	e.g. "B-2024-001"
[bill.patientName]	[customer_name] (contact lookup)	Auto-creates contact if missing
[bill.date]	[date]	ISO format YYYY-MM-DD
[bill.totalAmount]	sum of [line_items[].amount]	
[item.description]	[line_items[].name]	e.g. "Consultation", "Surgery"
[item.amount]	[line_items[].rate] (qty=1)	
[item.ledgerName]	[line_items[].account_id]	Mapped from account mapping table
["Bill #X - PatientName"]	[notes]	Narration field

Zoho API call: [POST /invoices]

2. Payment Received -> Zoho Books Customer Payment

Trigger: Payment recorded ([pushPaymentToZoho])

HMS Field	Zoho Books Field	Notes
[payment.receiptNumber]	[reference_number]	
[payment.patientName]	[customer_id] (contact lookup)	
[payment.date]	[date]	

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

[payment.amount]	[amount]	
[payment.paymentMode]	[payment_mode]	Cash->[cash], UPI/Card/NEFT->[bank_transfer]
linked bill's zoho_id	[invoices[].invoice_id]	Links payment to invoice
bank ledger name	[account_id]	From payment mode mapping

Zoho API call: [POST /customerpayments]

3. ESIC Bill -> Zoho Books Sales Invoice

Trigger: ESIC bill created ([pushESICBillToZoho])

HMS Field	Zoho Books Field	Notes
[bill.billNumber]	[reference_number]	
["ESIC"] (fixed contact)	[customer_id]	Single ESIC contact in Zoho
[bill.esicNumber]	appended to [notes]	"ESIC# IP-123"
[bill.serviceType]	[line_items[].name]	"ESIC IPD", "ESIC OPD", etc.
[bill.items[].amount]	[line_items[].rate]	
ESIC income account	[line_items[].account_id]	From mapping: "ESIC Income - IPD"

Difference from standard bill: Contact is always "ESIC" not the patient name.

4. Insurance Claim Bill -> Zoho Books Sales Invoice (split)

Trigger: Insurance bill ([pushInsuranceBillToZoho])

This is the most complex sync -- two debtors for one bill:

HMS Field	Zoho Books Field	Notes
[bill.billNumber]	[reference_number]	
[bill.insuranceCompany]	[customer_id]	Insurance company contact
[bill.claimAmount]	Invoice total	Amount insurance owes
[bill.patientShare]	Second invoice total	Separate invoice for patient share
[bill.policyNumber]	appended to [notes]	
[bill.tpaName]	appended to [notes]	
[bill.items[].description]	[line_items[].name]	Service descriptions

Strategy: Create two invoices in Zoho Books:

- Invoice 1: Insurance company contact, [claimAmount], account = Insurance Receivables
- Invoice 2: Patient contact, [patientShare], account = Hospital Income

5. Insurance Settlement -> Zoho Books Customer Payment (with adjustments)

Trigger: TPA/insurance payment received ([pushInsurancePaymentToZoho])

HMS Field	Zoho Books Field	Notes
[payment.receiptNumber]	[reference_number]	
[payment.insuranceCompany]	[customer_id]	

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

[payment.amount]	[amount]	Net amount received
[payment.utrNumber]	appended to [description]	Bank transfer reference
[payment.bankAccount]	[account_id]	Bank account in Zoho
[payment.tdsAmount]	Credit Note amount	TDS withheld -> separate credit note
[payment.disallowanceAmount]	Credit Note amount	Disallowed claims -> separate credit note

Strategy: One Customer Payment + optional Credit Notes for TDS and disallowance.

6. Pharmacy Sale -> Zoho Books Sales Invoice

Trigger: Pharmacy direct sale ([pushPharmacySaleToZoho])

HMS Field	Zoho Books Field	Notes
[sale.invoiceNumber]	[reference_number]	
[sale.patientName]	[customer_id]	Auto-create contact if missing
[sale.date]	[date]	
[item.medicineName]	[line_items[].name]	Medicine name as line item
[item.quantity]	[line_items[].quantity]	
[item.amount / qty]	[line_items[].rate]	Unit price
Pharmacy Sales account	[line_items[].account_id]	From account mapping

7. Contact Auto-Creation

Before creating any invoice/payment, the system checks [zoho_books_contact_mapping]:

- Hit: use stored [zoho_contact_id]
- Miss: call [POST /contacts] to create, store in mapping table

Contact Type	Zoho [contact_type]	Name
Patient	[customer]	Patient full name
Insurance company	[customer]	Company name e.g. "Star Health"
ESIC	[customer]	"ESIC" (one shared contact)
CGHS	[customer]	"CGHS"
Vendor/supplier	[vendor]	Supplier name

Payment Mode -> Zoho [payment_mode] Mapping

HMS Mode	Zoho [payment_mode]
Cash / CASH	[cash]
UPI	[bank_transfer]
Card / CARD	[creditcard]
NEFT / RTGS / ONLINE	[bank_transfer]
Cheque / DD	[check]
Insurance	(applied to insurance invoice)

Implementation Plan

Phase 1: Backend API Route

File: [api/zoho-books-proxy.ts]

Actions:

- [test-connection] -- verify OAuth token, fetch org info
- [push] -- create/update invoices, payments, contacts
- [sync] -- pull contacts and chart of accounts from Zoho Books
- [refresh-token] -- exchange refresh token for new access token

Auth: OAuth 2.0 -- store [access_token] + [refresh_token] + [expires_at] in [zoho_books_config]. Auto-refresh when within 5 min of expiry.

Phase 2: Service Library

File: [src/lib/zoho-books-service.ts]

Functions mirroring [tally-xml-service.ts]:

- [createContact(patient | insurer | vendor)]
- [createInvoice(bill)]
- [recordPayment(payment)]
- [getContacts()], [getInvoices()]
- [voidInvoice(zohoInvoiceId)]

Base URL: [https://www.zohoapis.in/books/v3] (India DC)

Phase 3: Auto-Push

File: [src/lib/zoho-books-auto-push.ts]

Mirror of [src/lib/tally-auto-push.ts]:

- [pushBillToZoho(billId)] -- OPD/IPD bills -> Sales Invoice
- [pushPaymentToZoho(paymentId)] -- Receipts -> Customer Payment
- [pushESICBillToZoho(billId)] -- ESIC bills
- [pushInsuranceBillToZoho(billId)] -- Insurance claims (split ledger)
- [pushInsurancePaymentToZoho(paymentId)] -- Settlement with TDS
- [pushPharmacySaleToZoho(saleId)] -- Pharmacy invoices

Fire-and-forget pattern (same as Tally): failures go to retry queue.

Phase 4: Retry Queue

Reuse: [tally_push_queue] table -- add a [source] column ([tally] | [zoho_books]) to distinguish entries.

File: [src/lib/zoho-books-retry.ts] -- thin wrapper around existing [tally-retry-processor.ts] logic.

Phase 5: Database Tables

New SQL migration file: [supabase/migrations/YYYYMMDD_create_zoho_books_tables.sql]

```
-- OAuth config & org settings
CREATE TABLE zoho_books_config (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id text NOT NULL,
  client_id text NOT NULL,
  client_secret text NOT NULL,
```

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

```
access_token text,  
refresh_token text NOT NULL,  
token_expires_at timestampz,  
is_active boolean DEFAULT true,  
last_sync_at timestampz,  
metadata jsonb  
);
```

-- Contact mapping: Adamrit entity -> Zoho Books contact_id

```
CREATE TABLE zoho_books_contact_mapping (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_type text NOT NULL, -- 'patient' | 'insurer' | 'vendor'  
  entity_id uuid NOT NULL,  
  zoho_contact_id text NOT NULL UNIQUE,  
  zoho_contact_name text,  
  created_at timestampz DEFAULT now()  
);
```

-- Invoice/payment mapping (prevent duplicate pushes)

```
CREATE TABLE zoho_books_transaction_mapping (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_type text NOT NULL, -- 'bill' | 'payment' | 'pharmacy_sale'  
  source_id uuid NOT NULL,  
  zoho_id text NOT NULL, -- zoho invoice_id or payment_id  
  zoho_number text, -- human-readable invoice number  
  status text DEFAULT 'synced',  
  synced_at timestampz DEFAULT now(),  
  UNIQUE(source_type, source_id)  
);
```

-- Sync log (same shape as tally_sync_log)

```
CREATE TABLE zoho_books_sync_log (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  sync_type text,  
  direction text, -- 'outward' | 'inward'  
  status text,  
  records_synced int DEFAULT 0,  
  records_failed int DEFAULT 0,  
  error_details jsonb,  
  started_at timestampz DEFAULT now(),  
  completed_at timestampz  
);
```

Phase 6: Chart of Accounts Mapping

File: [src/config/zoho-books-ledger-map.ts]

Maps Adamrit income categories -> Zoho Books account names (same as [tally_ledger_mapping]):

```
Pharmacy Sales -> "Pharmacy Sales"  
ESIC Income    -> "ESIC Receivables"  
Cash           -> "Cash"  
HDFC Bank      -> "HDFC Bank Current Account"
```

Stored in [zoho_books_config.metadata] JSONB, editable via UI.

Phase 7: UI Dashboard

Directory: [src/components/zoho-books/]

Components (mirror of [src/components/tally/]):

- [ZohoBooksPage.tsx] -- tabs router
- [ZohoBooksDashboard.tsx] -- OAuth connect, sync status, stats
- [ZohoBooksInvoices.tsx] -- list synced invoices
- [ZohoBooksContacts.tsx] -- list synced contacts
- [ZohoBooksMapping.tsx] -- ledger/account mapping editor
- [ZohoBooksBillSync.tsx] -- manual bill push
- [ZohoBooksRetryQueue.tsx] -- failed push viewer

Add tab to existing accounts/settings page.

Phase 8: OAuth Setup Flow

Zoho Books requires one-time browser-based OAuth consent:

1. User clicks "Connect Zoho Books" in dashboard
2. Redirect to Zoho OAuth URL with [scope=ZohoBooks.fullaccess.all]
3. Zoho redirects back to [/api/zoho-books-proxy?action=oauth-callback]
4. Exchange code for tokens -> store in [zoho_books_config]

PART B: Bank API Integration -- SBI & Canara Bank

What the bank integration does

- Fetch live account balance from SBI/Canara bank accounts
- Fetch bank statement (last 30 days) automatically
- Auto-reconcile bank transactions against HMS payment records
- Show matched/unmatched entries in the existing [TallyBankReconciliation] UI

Bank API Options (India)

Option 1: SBI YONO Business API (Recommended for SBI)

- Portal: [apihub.yonobusiness.sbi]
- Auth: OAuth 2.0 (Client ID + Secret Key per application)
- Registration: Corporate must register at the portal; SBI approves and activates
- Available APIs:
 - [GET /balance] -- account balance + hold amounts
 - [GET /statement] -- transactions for a date range (max 30 days, formats: PDF/XLS/text)
 - [POST /payment] -- fund transfers (single + bulk)
- Security: IP whitelisting required + OTP 2FA for transactions

Option 2: Canara Bank API Banking

- Portal: Apply via Canara Bank API Banking Application Form (corporate)
- Auth: OAuth 2.0 + multi-factor authentication
- Available APIs:
 - Account balance
 - Transaction history

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

- Payment initiation
 - Plans: Basic / Standard / Advanced (different TPS limits)

Option 3: RBI Account Aggregator Framework (Universal fallback)

- Works with ALL Indian banks including SBI and Canara -- no separate registration per bank
- Licensed AAs: Setu ([setu.co]), OneMoney, FinVu
- How it works: User consents -> AA fetches from bank -> returns structured JSON
- Best for: Multi-bank support, no direct bank API agreements needed
- Use if: Direct bank API approval takes too long

Step-by-Step Implementation Plan

PREREQUISITE A -- Get SBI Bank API Access (Do This First)

SBI provides bank API access through the YONO Business API Hub. Here is the exact process:

Step A1: Register on SBI API Hub

1. Go to [apihub.yonobusiness.sbi]
2. Click Sign Up (top right)
3. Fill in: Organisation Name, Contact Person, Email, Phone, GSTIN (optional), Nature of Business
4. Select Product Category: Corporate Banking / ERP Integration
5. Submit. SBI will send an activation link to your registered email within 1-2 business days.

Step A2: Create an Application

1. After activation, log in and go to My Applications -> Create New Application
2. App Name: e.g. "Adamrit HMS Integration"
3. Select APIs to subscribe: Account Balance, Account Statement, Fund Transfer (optional)
4. Submit for approval.

Step A3: Get Credentials

1. Once approved, SBI assigns:

- [Secret Key] (like a password)

2. Download credentials from the portal.

3. IP Whitelisting: Submit your server's IP address to SBI -- they will only allow API calls from that IP. For Vercel, you may need a fixed proxy IP.

Step A4: Sandbox Testing

1. Use sandbox base URL: [https://apihub.yonobusiness.sbi/sandbox/v1/]
2. Test balance and statement APIs with sandbox account numbers.
3. Once tested, request Production Access from SBI.

Step A5: Production Go-Live

1. Submit production request with your account number(s)
2. SBI enables live access -- switch base URL to [https://apihub.yonobusiness.sbi/v1/]

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

PREREQUISITE B -- Get Canara Bank API Access (Do This First)

Step B1: Download & Fill Application Form

1. Get the Canara Bank API Banking Corporate Application Form from your nearest Canara Bank branch or the bank's official website
2. Fill in: Company Name, Account Number(s), Contact Person, Purpose of API integration, Technical Contact Email

Step B2: Submit to Your Bank Branch

1. Submit the form with:
 - Company registration documents
 - KYC documents of authorised signatory
2. Submit to your assigned Canara Bank relationship manager or the branch where the current account is held.

Step B3: Receive API Credentials

1. Canara Bank's API Banking team will contact you (typically 5-10 business days)
2. They provide:
 - [client_id]
 - [client_secret]
 - Corporate user credentials for OAuth flow
3. IP whitelisting may also be required -- provide your server IP.

Step B4: Integration & Testing

1. Canara Bank provides test credentials for their sandbox/UAT environment
2. Test account balance and statement APIs
3. Get production credentials after UAT sign-off

PREREQUISITE C -- Zoho Books Setup (Do This First)

Step C1: Create Zoho Books Account

1. Sign up at [zoho.com/in/books] with your hospital's GST number
2. Complete company setup: name, GSTIN, financial year start

Step C2: Register an API Application

1. Go to [api-console.zoho.in] (India DC)
2. Click Add Client -> choose Server-based Applications
3. Fill:
 - Homepage URL: your Vercel app URL
 - Authorized Redirect URI: [https://your-app.vercel.app/api/zoho-books-proxy?action=oauth-
4. Get [Client ID] and [Client Secret]

Step C3: Enable Required Scopes

- Scopes needed:
- [ZohoBooks.invoices.CREATE]

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

- [ZohoBooks.invoices.READ]
- [ZohoBooks.contacts.CREATE]
- [ZohoBooks.contacts.READ]
- [ZohoBooks.customerpayments.CREATE]
- [ZohoBooks.customerpayments.READ]
- [ZohoBooks.settings.READ]

Step C4: First OAuth Connection

- Once the app is built, user clicks "Connect Zoho Books" -> browser opens Zoho consent screen -> approve -> tokens saved automatically.

PREREQUISITE D -- Environment Variables to Set in Vercel

Once all credentials are obtained, add these to your Vercel project:

```
# Zoho Books (stored in DB, but initial setup via env)
```

```
ZOHO_CLIENT_ID=your_client_id
```

```
ZOHO_CLIENT_SECRET=your_client_secret
```

```
# SBI YONO Business
```

```
SBI_CLIENT_ID=your_sbi_client_id
```

```
SBI_SECRET_KEY=your_sbi_secret_key
```

```
# Canara Bank
```

```
CANARA_CLIENT_ID=your_canara_client_id
```

```
CANARA_CLIENT_SECRET=your_canara_client_secret
```

After first connect, tokens are stored encrypted in the database -- env vars only needed for initial auth.

STEP 0 -- Create Documentation File

File to create: [docs/zoho-books-bank-integration.md]

Copy the full plan (architecture, data mappings, bank API details, step-by-step guide, verification steps) into this file so the team has a reference doc inside the repo.

STEP 1 -- Database Migration

File to create: [supabase/migrations/20260526000000_create_zoho_books_tables.sql]

Creates:

- [zoho_books_config] -- OAuth tokens + org info
- [zoho_books_contact_mapping] -- patient/insurer -> Zoho contact_id
- [zoho_books_transaction_mapping] -- bill/payment -> Zoho invoice/payment id (prevents duplicates)
- [zoho_books_sync_log] -- audit trail
- [bank_api_config] -- SBI/Canara credentials per account
- [bank_statements] -- cached bank transactions
- [bank_reconciliation_matches] -- matched/unmatched entries
- ALTER [tally_push_queue] ADD COLUMN [source text DEFAULT 'tally']

STEP 2 -- TypeScript Types

Files to create:

- [src/types/zoho-books.ts] -- ZohoBooksConfig, ZohoBooksInvoice, ZohoBooksContact, ZohoBooksPayment
- [src/types/bank-api.ts] -- BankConfig, BankTransaction, BankStatement, ReconciliationMatch

STEP 3 -- Zoho Books API Proxy (Backend)

File to create: [api/zoho-books-proxy.ts]

Endpoints:

- [?action=oauth-callback] -- receive OAuth code, exchange for tokens, store in DB
- [POST body.action=test-connection] -- verify token, fetch org name
- [POST body.action=push] -- create invoice / payment / contact
- [POST body.action=sync] -- pull contacts + chart of accounts from Zoho
- [POST body.action=refresh-token] -- refresh expired access token

Token auto-refresh: if [token_expires_at] is within 5 min, refresh before every API call.

Pattern: mirrors [api/tally-proxy.ts] but uses [fetch] to Zoho REST API instead of XML to Tally.

STEP 4 -- Zoho Books Service Library

File to create: [src/lib/zoho-books-service.ts]

Base URL: <https://www.zohoapis.in/books/v3>

Auth header: Authorization: Zoho-oauthtoken {access_token}

Org param: ?organization_id={org_id}

Functions:

- [createOrGetContact(name, type)] -- checks mapping table first, creates if missing
- [createInvoice(bill)] -- POST /invoices
- [voidInvoice(invoiceId)] -- POST /invoices/{id}/status/void
- [recordCustomerPayment(payment)] -- POST /customerpayments
- [getInvoices(dateFrom, dateTo)] -- GET /invoices
- [getContacts()] -- GET /contacts
- [getChartOfAccounts()] -- GET /chartofaccounts

STEP 5 -- Zoho Books Auto-Push

File to create: [src/lib/zoho-books-auto-push.ts]

Mirrors [src/lib/tally-auto-push.ts] exactly, but calls [/api/zoho-books-proxy] instead of [/api/tally-proxy].

Functions (one per event type):

1. [pushBillToZoho(bill)] -- OPD/IPD bill -> Sales Invoice
2. [pushPaymentToZoho(payment)] -> Customer Payment
3. [pushESICBillToZoho(bill)] -> Sales Invoice (ESIC contact)
4. [pushInsuranceBillToZoho(bill)] -> two Sales Invoices (insurer + patient)
5. [pushInsurancePaymentToZoho(payment)] -> Customer Payment + Credit Notes
6. [pushPharmacySaleToZoho(sale)] -> Sales Invoice

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

Each function: try push -> on fail -> [enqueueForRetry(source='zoho_books')].

STEP 6 -- Retry Processor (Zoho)

File to create: [src/lib/zoho-books-retry.ts]

Thin wrapper: reads [tally_push_queue WHERE source='zoho_books'], replays via [/api/zoho-books-proxy]. Same exponential backoff as Tally retry.

Modify: [src/lib/tally-retry-processor.ts] -- filter by [source='tally'] so the two queues don't cross.

STEP 7 -- SBI Bank API Proxy (Backend)

File to create: [api/bank-proxy.ts]

Actions:

- [test-connection] -- validate SBI/Canara credentials
- [get-balance] -- fetch live balance for configured account
- [get-statement] -- fetch transactions for date range (max 30 days)
- [reconcile] -- compare fetched transactions against HMS payment records

SBI YONO Business API calls:

POST https://apihub.yonobusiness.sbi/v1/statement

Auth: Client-Id + Secret-Key headers + OAuth token

Canara Bank API calls:

POST https://[canara-api-endpoint]/accounts/statement

Auth: OAuth 2.0 bearer token

Credentials (client_id, secret, account_number) stored in [bank_api_config] table -- never in code.

STEP 8 -- Bank Reconciliation Service

File to create: [src/lib/bank-reconciliation-service.ts]

Functions:

- [fetchAndCacheStatement(bankConfigId, fromDate, toDate)] -- calls bank API, stores in [bank_statements]
- [runAutoReconcile(bankConfigId, date)] -- matches bank transactions to HMS payments by amount + date $\pm$ 1 day
- [getUnmatchedBankEntries(bankConfigId)] -- entries in bank not in HMS
- [getUnmatchedHmsEntries(date)] -- payments in HMS not seen in bank

Matching logic: exact amount match + date within $\pm$ 1 day + payment mode matches bank description.

STEP 9 -- Zoho Books UI Components

Directory to create: [src/components/zoho-books/]

Component	What it shows
-----------	---------------

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

[ZohoBooksPage.tsx]	Tab router (Dashboard / Invoices / Contacts /
[ZohoBooksDashboard.tsx]	OAuth connect button, connection status, last
[ZohoBooksInvoices.tsx]	Table of synced invoices with status
[ZohoBooksContacts.tsx]	Table of synced contacts
[ZohoBooksMapping.tsx]	Edit income account -> Zoho account name mapp
[ZohoBooksBillSync.tsx]	Manually push a bill by bill number
[ZohoBooksRetryQueue.tsx]	List failed pushes with retry/delete buttons

STEP 10 -- Bank Integration UI Components

Directory to create: [src/components/bank-integration/]

Component	What it shows
[BankIntegrationPage.tsx]	Tab router
[BankDashboard.tsx]	Live balance for each configured account, las
[BankStatementView.tsx]	Table of fetched transactions with matched/un
[BankReconciliationView.tsx]	Side-by-side: HMS payments vs bank transactio
[BankConfig.tsx]	Add/edit bank credentials (SBI or Canara), te

STEP 11 -- Wire into Routes & Sidebar

Modify: [src/components/AppRoutes.tsx]

- Add [lazy(() => import('../pages/ZohoBooksIntegration'))] at [/zoho-books]
- Add [lazy(() => import('../pages/BankIntegration'))] at [/bank-integration]

Modify: [src/components/sidebar/menuItems.ts]

- Add "Zoho Books" below Tally Integration
- Add "Bank Integration" below Zoho Books

Create: [src/pages/ZohoBooksIntegration.tsx] (wrapper)

Create: [src/pages/BankIntegration.tsx] (wrapper)

Critical Files Summary

Action	File
Create	[supabase/migrations/20260526000000_create_zo
Create	[src/types/zoho-books.ts]
Create	[src/types/bank-api.ts]
Create	[api/zoho-books-proxy.ts]
Create	[api/bank-proxy.ts]
Create	[src/lib/zoho-books-service.ts]
Create	[src/lib/zoho-books-auto-push.ts]
Create	[src/lib/zoho-books-retry.ts]

Adamrit HMS - Zoho Books + SBI & Canara Bank Integration Plan

Create	[src/lib/bank-reconciliation-service.ts]
Create	[src/components/zoho-books/] (7 files)
Create	[src/components/bank-integration/] (5 files)
Create	[src/pages/ZohoBooksIntegration.tsx]
Create	[src/pages/BankIntegration.tsx]
Modify	[src/components/AppRoutes.tsx]
Modify	[src/components/sidebar/menuItems.ts]
Modify	[src/lib/tally-retry-processor.ts] (add [sour

Files to Reuse/Reference

- [api/tally-proxy.ts] -- proxy pattern
- [src/lib/tally-auto-push.ts] -- push function shapes
- [src/lib/tally-retry-processor.ts] -- retry logic
- [src/types/tally.ts] -- type shape reference
- [src/components/tally/TallyBankReconciliation.tsx] -- reconciliation UI pattern

Rate Limit & Security Notes

- Zoho Books: 100 req/min -> 700ms delay between queued pushes; retry on 429
- SBI YONO: IP whitelist required -- add Vercel edge function IPs to SBI whitelist
- Canara Bank: OAuth tokens expire -- store + auto-refresh same as Zoho
- All credentials stored in Supabase tables, never in [.env] or code

Verification Steps

Zoho Books

1. Register app in Zoho Developer Console -> get client_id + secret
2. Click "Connect Zoho Books" in dashboard -> complete OAuth in browser -> token saved
3. Finalize a test bill -> verify Sales Invoice appears in Zoho Books portal
4. Record a payment -> verify Customer Payment linked to invoice
5. Push same bill twice -> only one invoice exists (idempotency check)
6. Kill token -> bill push fails -> appears in retry queue -> restore token -> retry succeeds

SBI Bank API

1. Register at [apihub.yonobusiness.sbi] -> get Client ID + Secret
2. Add credentials in Bank Config screen -> click "Test Connection" -> shows account name
3. Click "Fetch Statement" -> last 30 days transactions appear in statement view
4. Record a payment in HMS for same amount/date -> auto-reconcile shows it as matched
5. Check unmatched entries for any bank credits not in HMS

Canara Bank API

1. Submit Canara Bank API Banking Application Form -> receive credentials
2. Same steps 2-5 as SBI above